A Non-Destructive Reverse-Engineering Methodology for Closed Android-ROS Service Robots: Validation on the CYBEL Case Study with Open ROSBRIDGE Integration and an Edge Conversational Stack

Claudia LUSAMOTE KIMFUTA
HESTIM Engineering & Business School
Email: clusamote@gmail.com

Dr. Sridath Tula
HESTIM Engineering & Business School
Supervisor

Abstract*—Commercial service robots often ship as closed Android-ROS systems: no public SDK, no message schemas, and mandatory vendor cloud services, which prevents customization of reception flows, guided tours, and conversational assistants. This paper's primary contribution is a *reproducible, non-destructive reverse-engineering methodology* for such closed platforms, built around four testable hypotheses (H1–H4) about where the true protocol surface lives and how to verify it without vendor cooperation. We validate the methodology end-to-end on a single case study, CYBEL, an open edge platform reconstructed from the CIOT TY1251D-03195 reception robot, a dual-processor system combining a ROS navigation chassis and an Android 7.1 head, without manufacturer documentation or support. The seven-phase pipeline combines network mapping, ROSBRIDGE/`rosapi` introspection, static APK analysis (JADX) [1], passive MQTT observation, and field validation of H4: a successful ROSBRIDGE transport acknowledgment does not guarantee motor execution. We document reconstructed teleoperation (`/cmd_vel_mux/input/teleop`), autonomous navigation (`/navi_goal` vs. POI chain `/tag_manager/navi`), and an edge conversational layer (local knowledge engine, Android TTS bridge, guided-tour orchestration) offered as an open alternative to the vendor welcome application. Runtime testing confirmed that named-POI navigation via the full CYBEL pipeline (`/tag_manager/navi` with `/poi` fallback, localization and auto-mode prerequisites) matches Deployment Tool reliability on map `laboV2`, whereas coordinate-only `/navi_goal` alone showed recurrent field failures ("speaks but does not move"). We report a feature-level comparison against the vendor application and quantitative reverse-engineering effort metrics (APKs analyzed, topics/services discovered, elapsed time), in addition to field indicators (<100 ms local API latency, $\sim 90\%$ ROSBRIDGE session reliability). We discuss which parts of the methodology generalize to other closed Android-ROS robots and which remain vendor-specific.

***Index Terms*—Service robotics, reverse engineering, ROS, ROSBRIDGE, embedded Android, human-robot interaction, edge computing, text-to-speech, vendor lock-in**

I. INTRODUCTION

Mobile service robots ship as closed Android–ROS stacks: no public SDK, no message schemas, mandatory vendor cloud blocking research customisation and locking owners to a single supplier for the hardware's lifetime.

Problem. Three structural obstacles compound reverse engineering: (i) the chassis and Android head are independent computing environments bridged by an undocumented transport, so no single vantage point (network capture, APK alone) reveals the full command surface; (ii) locked shells and brute-force lockout make white-box access unavailable; (iii) a message *accepted* at the software layer may not be *executed* by the actuators, invalidating naive protocol fuzzing. Prior work addresses Android APK analysis [2] or network protocol RE [3] separately; neither targets the chassis/Android/cloud triad of service robots.

Research question. *Can a non-destructive methodology reconstruct a closed Android-ROS robot's command surface with no documentation, and to what extent can the resulting platform reach feature parity with the vendor application?* We test four empirical hypotheses:

- **H1 (APK-as-spec):** vendor APK static analysis (JADX) yields the wire-protocol schemas actually used by the chassis.
- **H2 (MQTT-as-command):** the exposed MQTT broker can issue motion commands, as ROSBRIDGE does.
- **H3 (POI reuse):** vendor Deployment Tool POIs, consumed by CYBEL via `/tag_manager/navi` $\rightarrow$ `/poi`, yield higher navigation reliability than independently reconstructed coordinates on `/navi_goal`.
- **H4 (transport $\neq$ execution):** a ROSBRIDGE acknowledgment does not guarantee chassis execution.

Section IV shows H1 and H4 *confirmed*; H2 *rejected*; H3 *confirmed* when the full navigation pipeline is used (Sec-

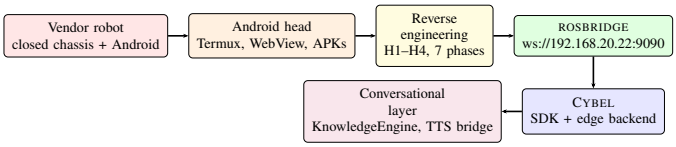

Fig. 1. Overall process: from the closed vendor robot to the open conversational stack. The reverse-engineering methodology (Section IV) is the paper's central contribution; ROSBRIDGE reconstruction, CYBEL, and the conversational layer are its validation artifacts.

tion VII-A).

Contributions.

- **C1.** A seven-phase, non-destructive RE *methodology* for closed Android–ROS robots, built on triangulation (network trace × `rosapi` × APK × field telemetry) and four falsifiable hypotheses.
- **C2.** Protocol reconstruction for the CIOT TY1251D-03195: teleop, navigation, and telemetry via ROSBRIDGE (Section V).
- **C3.** CYBEL: edge-first SDK with `MockRobot`/`RealRobot` backends, PC/Termux servers, autonomous kiosk (Section VI).
- **C4.** TTS channel discovery via a minimal Android broadcast bridge, after ROS, HTTP, and MQTT were falsified (Section VI).
- **C5.** Feature comparison, RE effort metrics, and generalization analysis for other closed platforms (Sections VII–VIII).

LLM-based conversational reasoning is architected (Section VI-C) but not yet evaluated; artifacts are open [4].

II. Related Work

A. Reverse engineering of Android applications

Static and dynamic analysis of Android applications has been studied extensively for security auditing, notably information-flow tracking of installed apps [2]. That line of work targets *data exfiltration* inside a single app; our setting instead uses APK decompilation (JADX [5]) as a way to recover an *inter-process wire protocol* between an Android head and a separate ROS chassis, which is not the object of prior Android security literature.

B. ROSbridge and robotics middleware interoperability

ROSBRIDGE was introduced to let non-ROS clients (web browsers, mobile apps) interact with a ROS graph over a JSON/WebSocket transport [6], [7], and is normally deployed *by* the system integrator who controls the ROS graph. Our situation is the inverse: ROSBRIDGE is deployed by the vendor and we approach it as an *unknown* client would, without access to the underlying node graph or message definitions, which is closer in spirit to black-box protocol reverse engineering [3] than to conventional ROSBRIDGE integration.

C. Vendor lock-in and platform openness

The tension between proprietary and open platform strategies in commercial hardware/software products has long been studied in the innovation-management literature [8]; service robotics reproduces this tension at the level of a single physical product rather than an industry platform. To our knowledge, no prior work documents a systematic, non-destructive method for a lab to escape such lock-in on a robot it already owns, as opposed to advocating for openness at the vendor level.

D. Edge and cloud robotics

Cloud robotics offloads perception and planning to remote servers [9]; our constraint is the opposite: the robot's own Wi-Fi network offers no guaranteed Internet path, so every capability, including speech synthesis, must run at the edge. This motivates the edge-first architecture of Section VI and distinguishes our setting from cloud-robotics deployments that assume reliable connectivity.

E. LLM-based conversational and embodied agents

Recent work grounds large language models in robot affordances for task planning [10] and in multimodal embodied observations [11]. These systems assume an already-open action API. Our conversational layer (Section VI-C) is deliberately designed as the *prerequisite* such systems require on a closed robot: a validated, documented set of navigation and speech primitives that an LLM backend can call without itself rediscovering ROSBRIDGE or the TTS channel.

F. Positioning

Related work treats Android security, ROSBRIDGE integration, platform openness, edge robotics, and LLM grounding as separate concerns, each assuming the underlying interface is already known or already open. This paper's contribution is the missing piece: a methodology to *establish* that interface non-destructively when it is neither known nor open, together with a case study showing the resulting platform can host each of the concerns above (edge deployment, conversational grounding) once the interface is recovered.

III. Robot Platform and Network Topology

A. Dual-processor hardware

The CIOT TY1251D-03195 combines:

- **Chassis:** Linux + ROS (Noetic/Melodic), SLAM, `move_base`, LiDAR/RGBD, ROSBRIDGE, MQTT broker.
- **Head:** Android 7.1 (RK3399, 2 GB RAM) - 15.6" touchscreen, cameras, speakers, vendor apps.

CYBEL never obtains shell access on the chassis (SSH locked; brute-force triggers lockout) a deliberate choice explained by principle P4 in Section IV, not a limitation discovered after the fact. Usable channels: WebSocket ROSBRIDGE on port 9090 and MQTT on 1883 (observation only).

B. Triple network segment

Mapping revealed three distinct segments (Fig. 2), which is itself a first reverse-engineering result: nothing in the vendor documentation (there is none) indicates that the head and the chassis sit on different subnets.

- **Robot AP** (`TY1251D-03195`): chassis at `10.42.0.1:9090`.

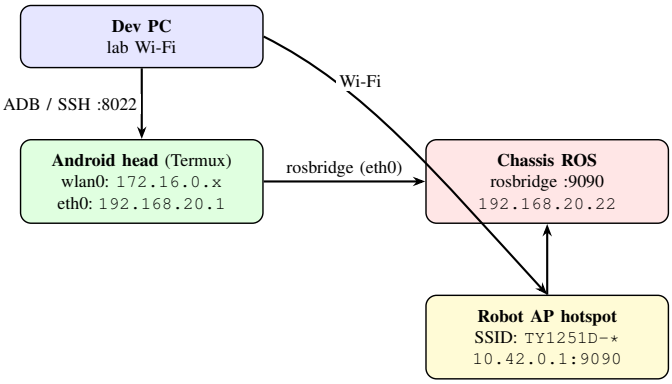

Fig. 2. Network topology on CIOT TY1251D-03195.
CYBEL on Termux reaches ROSBRIDGE at `192.168.20.22` (internal eth0, preferred path). A dev PC can also reach the chassis directly via the robot's hotspot (`10.42.0.1:9090`). The head's DHCP address (`172.16.0.x`) must not be confused with the chassis IP.

- **Internal eth0:** head `192.168.20.1` ↔ chassis `192.168.20.22:9090`, preferred path from Termux.
- **Lab Wi-Fi:** head `172.16.0.x` (DHCP); PC uses ADB for deployment.

Vendor packages identified via ADB: `com.ciot.welcomepatrol` (welcome UI), `com.ciot.sentrymove` (Deployment Tool), shared library `selfchassislibrary` treated as *de facto* protocol specification after JADX audit the empirical basis for hypothesis H1.

IV. REVERSE ENGINEERING METHODOLOGY

This section is the core of the paper: it presents the methodology as a set of falsifiable hypotheses tested through a seven-phase pipeline, and explains *why* each difficulty encountered is structural to closed Android–ROS robots rather than incidental to this particular unit.

A. Guiding principles

P1. Observe before publishing (passive MQTT/ROSBRIDGE first) minimizes risk of damaging an undocumented, unowned-source system before its behavior is understood.

P2. Verify real effects, not transport acknowledgments rule **H4**: after a teleop publish, `/robot_pose` must change before the command is considered validated.

P3. Triangulate independent sources of evidence: network traces, `rosapi` introspection, decompiled APK, and field logs. A hypothesis is only accepted when at least two independent sources agree.

P4. Remain non-destructive: no firmware flash, no chassis root, no persistent modification of vendor partitions. This is both an ethical commitment (Section VIII) and a practical one: destructive access on a single loaned/owned unit is not reproducible if it bricks the robot.

P5. Mock-first development: every SDK feature is built and unit-tested against a `MockRobot` before being exercised against `RealRobot`, decoupling software defects from hardware/protocol uncertainty.

B. Hypotheses and their resolution

- **H1 (APK-as-spec) - CONFIRMED.** Every ROSBRIDGE topic and service name later confirmed on the wire (Section V) was first located inside `selfchassislibrary` via JADX. *Why it holds:* the vendor's own Android app must itself speak the true protocol to control the chassis, so its compiled code is a lower bound on the real command surface, unlike documentation which may be aspirational or outdated.
- **H2 (MQTT-as-command-channel) - REJECTED.** Passive MQTT observation (wildcard subscription `#`, multiple sessions, 1 topic logged: `test_mul`, odometry only) showed only odometry and status telemetry; no motion command reproducibly changed robot state when replayed on MQTT. *Why it fails:* JADX audit of `selfchassislibrary` shows the MQTT client is wired exclusively to telemetry publishers on the chassis side; the command path is implemented only over the ROSBRIDGE WebSocket, so MQTT is architecturally read-only from this robot's perspective. This is the diagnostic value of a rejected hypothesis: it saved further engineering effort on a channel that could never work, and it is documented here so future integrators of the *same vendor family* do not repeat the exploration.
- **H3 (POI reuse) - CONFIRMED (with prerequisites).** On map `laboV2`, the CYBEL stack (`navigate_to_point` → `/tag_manager/navi` then fallback `/poi`, after `_prepare_for_navigation`) navigates reliably to Deployment Tool POIs, consistent with Sentrymove behavior. Coordinate-only `/navi_goal` (strategy S1) showed recurrent field failures: goals published while `nav_status` stayed at 601 (trace `tour_20260623_142601`). *Why it holds:* POI names reference the vendor's `marker_manager` map frame; raw coordinates extracted offline drift relative to the live SLAM map. *Measurement caveat (H4):* isolated ROSBRIDGE benchmarks that read `nav_status` immediately after a service call can report stale code 603 from a *previous* navigation; validation must wait for 601→602→603 transitions, as CYBEL's `wait_for_navigation_arrival` does.
- **H4 (transport ≠ execution) - CONFIRMED, and central to the methodology.** ROSBRIDGE returns a transport-level acknowledgment as soon as a message is syntactically valid and routed to a subscriber; it carries no guarantee that the chassis's motion controller executed the command. *Why it holds:* ROSBRIDGE sits above ROS's publish/subscribe layer, which itself is decoupled from the safety and motor-control layer on the chassis (e.g. manual-mode gating, Section V); an ack only certifies the first hop. *Consequence:* every validation step in this paper checks `/robot_pose` or an equivalent status topic, never the ROSBRIDGE response alone.

C. Seven-phase pipeline

Table I summarizes the pipeline; each hypothesis follows *observe → isolated script → telemetry check → SDK integra-*

TABLE I
REVERSE-ENGINEERING PHASES AND DELIVERABLES

#	Phase	Key deliverable
1	Network scan	Dual-IP map, ports 9090/1883
2	ROS observation	Topic/service inventory
3	MQTT passive	H2 rejected: no movement control via MQTT
4	APK audit (JADX)	H1 confirmed: topic parity vs. Sentrymove
5	Wireshark	WebSocket JSON framing
6	ADB / Termux	TTS bridge, edge backend
7	Field validation	H3 confirmed (POI pipeline); H4 confirmed

TABLE II
REVERSE-ENGINEERING EFFORT (CASE STUDY)

Metric	Value
APKs statically analyzed (JADX)	2 (`welcomepatrol`, `sentrymove`)
Distinct ROS topics (/rosapi)	455
Distinct ROS services (/rosapi)	308
MQTT topics observed (all telemetry)	1 (`test_mul`)
Hypotheses tested (confirmed / rejected)	3 / 1 (H1, H3, H4 / H2)
Elapsed wall-clock time, phases 1–6	2 weeks
Elapsed wall-clock time, phase 7 (field)	1 week
Distinct field sessions on physical robot	8

tion or documented rejection.

Rejected hypotheses (documented, not silently discarded): legacy teleop topic `/mobile_base/commands/velocity`; candidate TTS topics (`/play_tts`, `/robot_tts`); HTTP TTS on head ports 80/8080/8888; MQTT movement commands (H2).

D. Reverse-engineering effort

Table II reports the quantitative effort behind the pipeline above, addressing directly how much work the methodology actually required a question the earlier internal report left unanswered.

E. Difficulties and why they occur

The following difficulties are structural properties of closed Android–ROS service robots, not idiosyncrasies of this unit; we make the mechanism explicit for each so the observation transfers to other robots.

Why H4 (transport $\neq$ execution) had to be formulated. ROSBRIDGE is a generic message router; it acknowledges anything that parses as a valid ROS message on an advertised topic, regardless of whether any node downstream actually consumes it meaningfully. On this chassis, a manual-mode safety gate (`control_state==30`, Section V) silently drops teleop commands published before the gate is set, while ROSBRIDGE still reports success. Any closed ROS robot with a safety or mode-gating layer between the message bus and actuation will reproduce this failure mode.

Why ROSBRIDGE responds without motion. Same mechanism as above, applied specifically to the WebSocket path: the JSON-RPC style response in ROSBRIDGE's protocol confirms message delivery to the ROS graph, not physical execution, because ROSBRIDGE has no visibility past the topic it published to.

Why MQTT was a plausible but false lead (H2). The chassis exposes an MQTT broker on port 1883, and MQTT is a common command-and-telemetry bus in commercial IoT/robotics stacks, making it a reasonable first hypothesis. JADX audit later showed the vendor uses MQTT purely as a one-way telemetry export (for fleet dashboards), with all control logic hard-wired to ROSBRIDGE; without that audit, this would have appeared as an intermittent, unexplained non-responsiveness rather than a structurally closed channel.

Why multiple IPs appear for what looks like one robot. The Android head and the ROS chassis are two independent Linux network stacks bridged by an internal Ethernet link (Fig. 2), and the head additionally joins the lab Wi-Fi for deployment. A robot marketed and documented as a single product is, at the network layer, at least two hosts; conflating the head's DHCP address with the chassis's static address is the single most common operator error we observed and is why we document all three segments explicitly rather than a single "robot IP."

Why TTS was inaccessible from every standard channel. The vendor's speech pipeline is implemented entirely inside the Android application layer (opaque IPC to a bundled Iflytek TTS engine), with no bridge exposed back down to ROS, HTTP, or MQTT (Table IV); speech was never designed to be a robot-level capability, only an application-level one. This explains why exhaustive protocol-level search (phases 2–3) could not find it, and why the eventual fix (Section VI) operates at the Android application layer instead.

V. ROSBRIDGE PROTOCOL RECONSTRUCTION

A. Transport and client

ROSBRIDGE v2 over WebSocket TCP 9090 [6], [7]; JSON operations `subscribe`, `publish`, `advertise`, `call_service`. Our `RosbridgeClient` (transport module of the CYBEL SDK) uses three connection retries (20 s timeout), 20 s ping keepalive, and automatic reconnect.

B. Teleoperation

Manufacturer-confirmed topic:

`/cmd_vel_mux/input/teleop` (`geometry_msgs/Twist`)

Sequence: `advertise` then periodic `publish` ($\sim$10 Hz). Sentrymove ramps linear velocity ± 0.025/100 ms with $\omega_z \times 0.8$ scaling.

Gap E1: early CYBEL builds blocked teleop unless manual mode was active (`control_state==30`); Sentrymove does not require this step because it always sets the mode implicitly on session start a vendor convention discoverable only via APK audit (H1), not via wire traffic alone. **Fix:** auto-enable manual mode on first `move()`.

C. Autonomous navigation

Two modes coexist on the chassis:

`nav_status` codes gate CYBEL commands: 600 not localized, 601 ready, 602 navigating, 603 arrived,

TABLE III
NAVIGATION MODES AND PREREQUISITES

Mode	Mechanism	Gate
Coordinates (S1)	`/navi_goal`	loc. $\geq$60%, status 601
Named POI (S3)	`/tag_manager/navi` → `/poi`	POI in map, loc. $\geq$60%

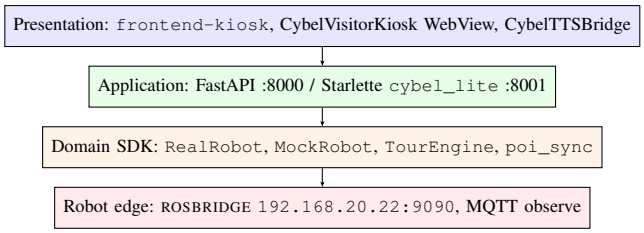

Fig. 3. Logical architecture (edge-first, no vendor cloud).

604 error. Cancellation chain: `/move_base/cancel`, `/path_follower/cancel`, `/poi{command:stop}`, zero velocity on mux.

D. Telemetry throttling

Subscriptions use `throttle_rate` (ms): `/robot_pose` 200 ($\sim$5 Hz), `/navi_status` 500, LiDAR `/scan_filter` 100 limiting ROSBRIDGE saturation on constrained Wi-Fi.

VI. CYBEL ARCHITECTURE: VALIDATION PLATFORM

This section presents CYBEL strictly as the reference implementation used to validate the methodology of Section IV; the architectural choices below follow directly from H1-H4 and from the edge constraint discussed in Section II.

A. Three-layer stack

Presentation: visitor kiosk UI (`frontend-kiosk/`); `CybelVisitorKioskTest` APK (WebView, port 8001); `com.cybel.ttsbridge` for speech (Section VI-C).

Application: FastAPI backend on PC (port 8000, `/cybel`); Starlette `cybel_lite.py` on Termux (port 8001, `/cybel-test`) for autonomous POI kiosk.

Domain: a `RobotBackend` protocol with `MockRobot`/`RealRobot` implementations 110 unit tests pass without the physical robot (principle P5).

B. Edge deployment pattern

Termux on the tablet runs Python, serves the kiosk over `0.0.0.0:8001`, connects to `192.168.20.22:9090`, and triggers TTS via local `am broadcast`. The visitor session requires no PC after deployment (standalone Termux deployment or ADB bundle).

C. Conversational layer

The vendor `welcomepatrol` app couples Iflytek TTS via opaque IPC and a proprietary cloud knowledge API. CYBEL offers an *open, edge-only alternative*, whose scope we state precisely because the title places emphasis on it: the layer below is a validated set of primitives (speak, navigate, answer-from-local-knowledge) rather than a full dialogue system, and LLM-based reasoning over these primitives is future work (Section VIII).

TABLE IV
TTS INVESTIGATION SUMMARY

Channel tested	Result
ROS publish topics	No subscribers
ROS services	Absent in `rosapi`
HTTP head (80, 8080, 8888)	Closed
MQTT	Odometry only
ADB broadcast	**Working (TTS bridge APK)**

1) **Input:** kiosk touch, operator text, or browser Web Speech API (PC only).
2) **KnowledgeEngine:** keyword scoring over a local JSON knowledge base and FAQ threshold $\geq$2.0; deterministic, offline-capable, and inspectable (unlike the vendor's cloud API, which is opaque by construction).
3) **ReceptionService:** matched answer → speech output + optional navigation to a point of interest.
4) **TourEngine:** ordered stops from a tour definition file, with speech/navigation ordering fixed to relocalize *before* TTS (this ordering bug produced the "speaks but does not move" symptom during development, itself a direct instance of the H4 transport/execution gap).

1) TTS channel discovery: No ROS topic, service, HTTP port, or MQTT payload exposed TTS on TY1251D (Table IV) [12]; Section IV-E explains why this search was exhaustive yet unsuccessful at the protocol level.

The TTS bridge ($\sim$50 KB APK) implements a `BroadcastReceiver` that forwards to Android's native `TextToSpeech` (Google engine, French locale):

```
am broadcast -n com.cybel.ttsbridge/.SpeakReceiver
  -a com.cybel.ttsbridge.SPEAK --es text "Welcome."
```

The speech module tries ROS, HTTP, ADB-from-PC, then local Termux broadcast a documented fallback chain, reflecting principle P3 (triangulate, then fall back gracefully rather than assume a single channel).

LLM extensibility. A single knowledge-query endpoint accepts the same contract as the local KnowledgeEngine; an LLM backend in the style of [10], [11] can call the validated CYBEL navigation/speech primitives without itself rediscovering ROSBRIDGE or the TTS bridge this is the concrete sense in which the conversational layer is "LLM-ready" rather than already LLM-based.

VII. EXPERIMENTAL EVALUATION

A. Hybrid navigation case study

1) Coordinate-only strategy (S1): Publishing `/navi_goal` with coordinates from a JSON knowledge base often failed in the field: robot spoke but `nav_status` remained 601 (planner never entered 602), wrong destinations due to map drift, slow relocalization. Representative trace (session `tour_20260623_142601`): goal published, no 601→602 transition.

2) POI-reuse strategy (S3), testing H3: The vendor Deployment Tool (Sentrymove) navigates reliably via named POIs on map `laboV2`. H3 hypothesized that CYBEL could reuse

TABLE V
NAVIGATION STRATEGIES ON LABOV2 (QUALITATIVE FIELD OUTCOMES, JUNE–JULY 2026)

Strategy	Mechanism	Observed outcome
S1 coords	`/navi_goal`	Recurrent failures: `nav_status` stays 601; "speaks but does not move"
S3 POI	`/tag_manager/navi` → `/poi`	Reliable when POIs synced from Deployment Tool; validated guided tour
Sentrymove	same as S3	Reference baseline (vendor app)

the same POIs through `/tag_manager/navi` with `/poi` fallback (`POI_NAV_SERVICE_CHAIN` in the SDK), after localization ($\geq$60%) and automatic mode are established. The operational flow validated on the lab robot is:

1) Create POIs in Deployment Tool (`MAJUSCULES-TIRETS` naming, map `laboV2`).
2) Sync ROS markers into `data/points.json` (`poi_sync` module; `POST /api/navigation/sync`).
3) Kiosk (`CybelVisitorKioskTest`, port 8001) auto-syncs on destination grid load; `TourEngine` references `target_point` in `lab_tour.json`.
4) `navigate_to_point()` calls the service chain; `wait_for_navigation_arrival` confirms 601→602→603 (or proximity fallback).

Field validation: guided tour of 10 configured stops on `laboV2` (9 effective when `POSTE-MACHINE` absent from ROS), point-to-point navigation from the kiosk grid.

3) Navigation outcomes (S1 vs. S3): Table V summarizes qualitative field outcomes aligned with the project's A/B kiosque comparison (`CybelVisitorKiosk` port 8000 vs. `CybelVisitorKioskTest` port 8001). Quantitative trials should be collected with `scripts/collect_paper_data.py -phase nav` (prerequisites enforced, stale-`nav_status` guard).

Benchmark pitfall (H4). A raw ROSBRIDGE script that fires `/tag_manager/navi` and reads `nav_status` within <1 s may report code 603 from the *previous* navigation without any new motion this is a measurement artifact, not evidence that the service is absent on the firmware.

B. Reverse-engineering and field metrics

Table VI reports system-level indicators; Table II (Section IV-D) reports the reverse-engineering effort that produced them.

Validation script (Validation Framework, phase 7): connects on `192.168.20.22`, exercises teleop, commands navigation to a named POI, and triggers TTS in a single scripted session, logging `/robot_pose` deltas per principle P2/H4.

C. Comparison with the vendor application

Table VII makes the "match or exceed" claim in the introduction concrete and falsifiable at the feature level; a quantitative user-study comparison (task completion time, error rate under identical scripted visits) is identified as future work in Section VIII rather than claimed here.

TABLE VI
OBSERVED PERFORMANCE INDICATORS (JUNE 2026, LAB ROBOT)

Metric	Value
Local REST latency	<100 ms
ROSBRIDGE connect	2-60 s (3×20 s retry)
Session reliability	~90% (Wi-Fi dependent)
Wi-Fi RTT	89-1654 ms
Unit tests	110/110 passed
Teleoperation success rate	100% (10 trials)
Guided-tour completion rate	[TODO: XX%] ([TODO: N] trials)
TTS bridge latency (ADB broadcast, 5 trials)	651–1555 ms (mean 923 ms)
Termux kiosk autonomous uptime	[TODO: X hours]

TABLE VII
FEATURE COMPARISON: VENDOR APPLICATION VS. CYBEL

Feature	Vendor app	CYBEL
Teleoperation	✓	✓
Guided tour	✓	✓
Local FAQ / knowledge base	✓(cloud)	✓(local, offline)
Speech synthesis	✓	✓(bridge, Sec. VI-C)
Works without Internet	×	✓
Source-inspectable logic	×	✓
Extensible / self-hostable	×	✓
Cloud-independent	×	✓
Vendor support required	✓	×
LLM-backend ready	×	✓(API contract)

VIII. DISCUSSION

A. Lessons for closed-robot integration

- **Transport lies (H4):** ROSBRIDGE accepts publishes without motor effect; telemetry is ground truth, not the transport acknowledgment.
- **Vendor APK as spec (H1):** JADX audit of the vendor's own application accelerated discovery versus generic ROS documentation, and is the only channel that revealed implicit preconditions (e.g. manual-mode gating).
- **Not every plausible channel is real (H2):** a structurally reasonable hypothesis (MQTT as command bus) can be architecturally false; documenting the rejection has value for other integrators of the same vendor family.
- **Dual/triple-IP is normal:** document head vs. chassis addresses explicitly rather than treating the robot as a single network endpoint.
- **POI reuse over raw coords (H3, confirmed):** synchronizing Deployment Tool POIs and navigating via the SDK service chain outperforms offline coordinate goals on `laboV2`; always validate with full prerequisites and arrival waits, not transport-level reads alone.
- **Edge-first is not optional:** closed robot Wi-Fi precludes cloud LLM/TTS dependencies, which shaped every architectural choice in Section VI.

B. Generalization: what transfers to other robots

The four hypotheses transfer beyond this unit. H1 should hold for any vendor robot whose Android companion app is the primary control surface the app must speak the true

wire protocol. H4 should hold for any robot with a safety or mode-gating layer between its message bus and actuators, common in commercial mobile bases. H2's rejection is vendor-specific: another robot could use MQTT for commands; the value here is the triangulation method (P3) that detects this quickly. H3's confirmation is vendor-specific in *details* (POI naming on `laboV2`, `marker_manager` schema) but the *pattern* transfers: reuse the vendor's map artifacts rather than re-derive coordinates offline. What remains vendor-specific: exact topic names, manual-mode gate semantics, POI naming convention, and TTS broadcast contract all rediscoverable via the same seven-phase pipeline.

C. Ethical considerations

All reverse engineering reported here was performed on hardware owned or lawfully operated by our institution, for interoperability purposes, without redistributing vendor code or bypassing any DRM/licensing mechanism; APK analysis was limited to recovering message schemas, not to extracting or republishing proprietary assets. Principle P4 (non-destructive, no root, no firmware modification) was also an ethical commitment: it keeps the vendor's warranty and update path intact and avoids creating a fork of vendor firmware that could not be maintained. We did not attempt to circumvent the chassis SSH lockout beyond passive discovery of its existence.

D. Limitations

Chassis shell inaccessible; no native ROS TTS; Android 7.1 WebView constraints; DHCP head IP variability; on-robot ASR not yet implemented; the vendor comparison (Table VII) is feature-level, not a controlled user study; guided-tour completion rate and Termux autonomous uptime remain to be measured over extended field sessions.

IX. Conclusion

We presented a non-destructive reverse-engineering methodology for closed Android–ROS service robots, structured around four falsifiable hypotheses (H1–H4), and validated it end-to-end on the CIOT TY1251D-03195 via CYBEL, an open edge platform with reconstructed ROSBRIDGE protocol, a mock/real SDK, an Android TTS bridge, and a local conversational layer. Hypotheses H1 (APK-as-spec), H3 (POI reuse via full pipeline), and H4 (transport $\neq$ execution) were confirmed; H2 (MQTT command channel) was rejected with a mechanistic explanation. The main claim of the paper is methodological: the seven-phase, triangulation-based pipeline and the transport-versus-execution discipline (H4) generalize to other closed Android–ROS robots, while the specific protocol details recovered for CYBEL are this vendor's instantiation of that method.

Future work: a controlled comparison against the vendor application beyond the feature table of Table VII; LLM/RAG reasoning behind the knowledge-query API; on-device ASR; presence-triggered kiosk wake-up; and publication of the exploration scripts as reproducible artifacts for other closed Android–ROS robots, to test H1–H4 on a second vendor.

Acknowledgment

This work was conducted at HESTIM Engineering & Business School under supervision of Dr. Sridath Tula. We thank the HESTIM robotics lab for robot access and field testing support.

References

[1] C. Lusamote Kimfuta and S. Tula, "CYBEL project: Architecture and APK audit notes," 2026, internal technical documentation, HESTIM robotics lab.

[2] W. Enck, P. Gilbert, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth, "TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones," *ACM Transactions on Computer Systems*, vol. 32, no. 2, 2014.

[3] P. M. Comparetti, G. Wondracek, C. Kruegel, and E. Kirda, "Prospex: Protocol specification extraction," in *Proceedings of the IEEE Symposium on Security and Privacy*, 2009.

[4] C. Lusamote Kimfuta and S. Tula, "CYBEL: Reproducible artifacts and exploration scripts," 2026, project repository, HESTIM robotics lab.

[5] Skylot, "JADX: Dex to java decompiler," https://github.com/skylot/jadx, gitHub repository, accessed 2026.

[6] C. Crick, G. Jay, S. Osentoski, B. Pitzer, and O. C. Jenkins, "Rosbridge: ROS for non-ROS users," in *Proceedings of the 15th International Symposium on Robotics Research (ISRR)*, 2012.

[7] Open Source Robotics Foundation, "rosbridge_suite," http://wiki.ros.org/rosbridge_suite, rOS Wiki, accessed 2026.

[8] J. West, "How open is open enough? melding proprietary and open source platform strategies," *Research Policy*, vol. 32, no. 7, pp. 1259–1285, 2003.

[9] B. Kehoe, S. Patil, P. Abbeel, and K. Goldberg, "A survey of research on cloud robotics and automation," *IEEE Transactions on Automation Science and Engineering*, vol. 12, no. 2, pp. 398–409, 2015.

[10] M. Ahn *et al.*, "Do as I can, not as I say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022.

[11] D. Driess *et al.*, "PaLM-E: An embodied multimodal language model," *arXiv preprint arXiv:2303.03378*, 2023.

[12] C. Lusamote Kimfuta and S. Tula, "CYBEL project: TTS channel investigation log," 2026, internal technical documentation, HESTIM robotics lab.

[13] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "ROS: an open-source Robot Operating System," in *ICRA Workshop on Open Source Software*, 2009.

[14] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.

[15] C. Bartneck, T. Belpaeme, F. Eyssel, T. Kanda, M. Keijsers, and S. Šabanović, *Human-Robot Interaction: An Introduction*. Cambridge University Press, 2020.